



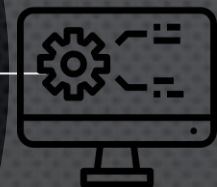
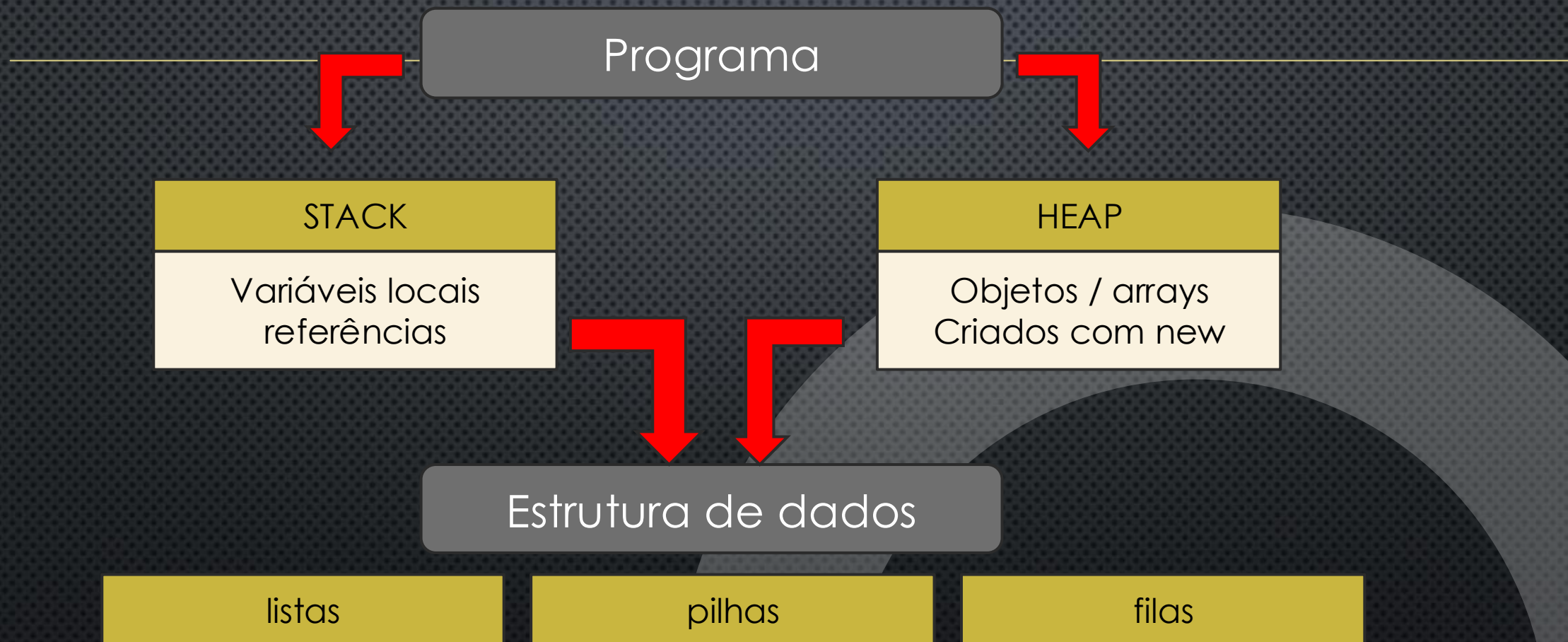
ESTRUTURA DE DADOS I

FUNDAMENTOS E ESTRUTURAS LINEARES

Prof Me Wesley Soares de Souza

✦
👤 Boas-vindas

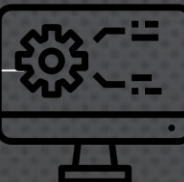
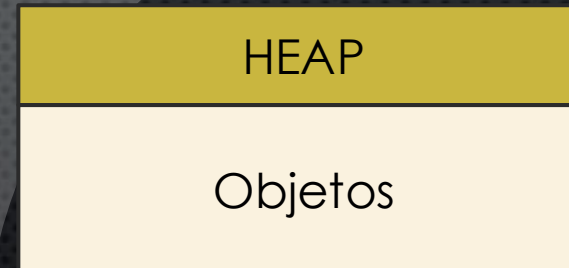
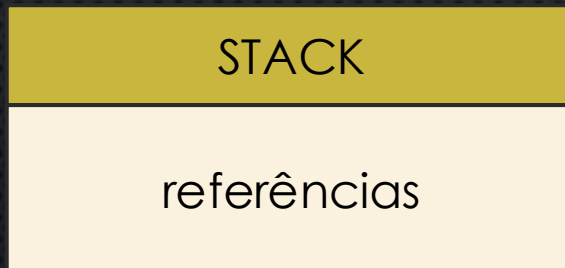
MEMÓRIA E ALOCAÇÃO DINÂMICA



ALOCAÇÃO DINÂMICA

Organizando dados em posições consecutivas

- **NA AULA ANTERIOR VIMOS:** `No N = NEW No(10);`
- **CRIANDO:** `10 → 20 → 30 → 40 → NULL`



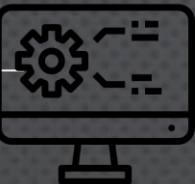
O QUE É UMA LISTA?

UMA LISTA É UMA COLEÇÃO DE ELEMENTOS ORGANIZADOS EM UMA DETERMINADA SEQUÊNCIA.

1. Ana
2. Carlos
3. João
4. Pedro
5. Maria

Podemos realizar operações como:

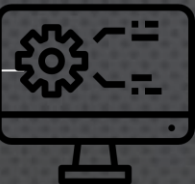
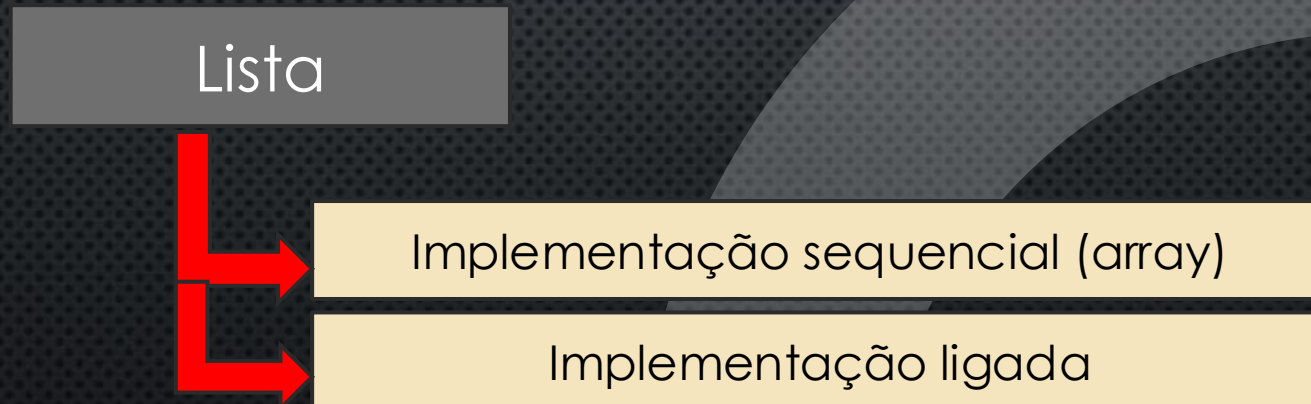
- inserir;
- remover;
- consultar;
- pesquisar;
- alterar;
- percorrer.



LISTA X ESTRUTURA DE ARMAZENAMENTO

LISTA: UMA SEQUÊNCIA DE ELEMENTOS.

ARRAY: É UMA ESTRUTURA DE ARMAZENAMENTO.



O QUE SIGNIFICA "SEQUENCIAL"?

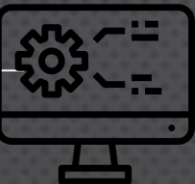
Na lista sequencial, os elementos são armazenados em posições consecutivas de um array.

```
int[] valores = new int[6];
```

10	20	30	null	null	null
0.	1.	2.	3.	4.	5

Complexidade?

$O(1)$



MAS EXISTE UM PROBLEMA...

10	20	30	40	50	60
0.	1.	2.	3.	4.	5

Quero inserir o valor 25 ??

Característica: Lista ordenada

10	20	25	30	40	50	60
0.	1.	2.	3.	4.	5.	6

Quantos elementos precisam ser deslocados? **N**

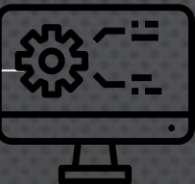
Qual a complexidade?

$O(n)$

A IDEIA DE LISTA LINEAR SEQUENCIAL

```
class ListaSequencial {  
  
    private int[] elementos;  
    private int tamanho;  
  
}
```

- **Elementos**: Onde os dados serão armazenados
- **Tamanho**: Quantos elementos estão realmente sendo utilizados



CAPACIDADE X TAMANHO

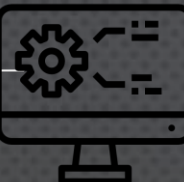
Sendo: `int[] elementos = new int[10];`

- Capacidade: • **10**
- Tamanho: • **4**

10	20	30	40	null	null	null	null	null	null
0.	1.	2.	3.	4.	5.	6.	7.	8.	9

Regra:

$0 \leq \text{tamanho} \leq \text{capacidade}$



CRIANDO A LISTA

```
public class ListaSequencial {
```

```
    private int[] elementos;
```

```
    private int tamanho;
```

Propriedades

```
    public ListaSequencial(int capacidade) {
```

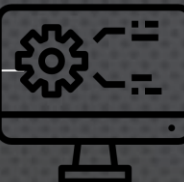
```
        elementos = new int[capacidade];
```

```
        tamanho = 0;
```

```
    }
```

```
}
```

Construtor



CRIANDO A LISTA

Crie uma lista com capacidade de 10 itens



INSERINDO ELEMENTO

```
public void adicionar(int valor) {  
  
    elementos[tamanho] = valor;  
    tamanho++;  
}
```

Chame o método adicionar na main e insira 3 valores

POR QUE USAMOS TAMANHO SE O VETOR POSSUI LENGTH?

Tamanho representa a **quantidade de elementos** armazenados.
Length representa a **capacidade total**.

PERCORRENDO A LISTA

```
public void imprimir() {  
  
    for (int i = 0; i < tamanho; i++) {  
        System.out.println(elementos[i]);  
    }  
}
```

Complexidade?

$O(n)$

CONSULTANDO POR POSIÇÃO

```
public int obter(int indice) {  
    return elementos[indice];  
}
```

Complexidade?

$O(1)$

VALIDANDO O ÍNDICE



- `lista.obter(20);`

```
public int obter(int indice) {  
  
    if (indice < 0 || indice >= tamanho) {  
        throw new IndexOutOfBoundsException();  
    }  
  
    return elementos[indice];  
}
```

Regra:

$0 \leq \text{tamanho} \leq \text{capacidade}$

PESQUISANDO UM ELEMENTO

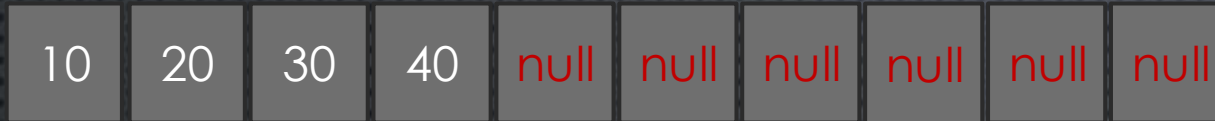
- Encontrar o valor 30.

Complexidade?

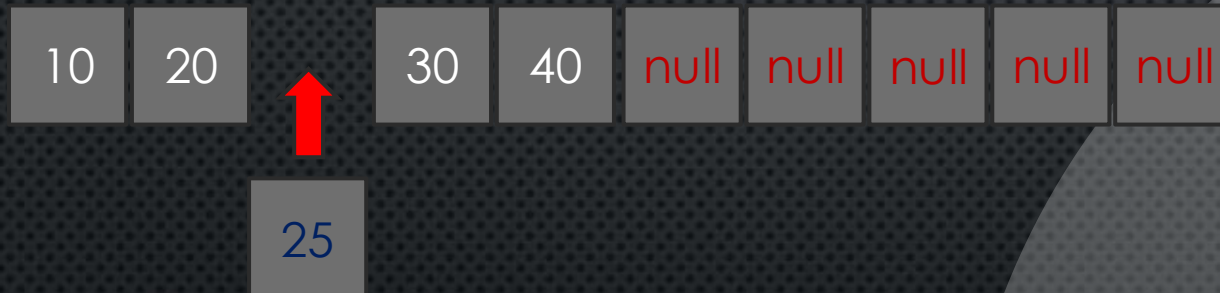
$O(n)$

```
public int buscar(int valor) {  
  
    for (int i = 0; i < tamanho; i++) {  
  
        if (elementos[i] == valor) {  
            return i;  
        }  
    }  
  
    return -1;  
}
```

INSERINDO NO MEIO



- Inserir o valor 25.



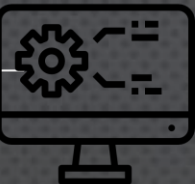

```
public void adicionar(int indice, int valor) {  
  
    if (indice < 0 || indice > tamanho) {  
        throw new IndexOutOfBoundsException();  
    }  
  
    if (tamanho == elementos.length) {  
        throw new IllegalStateException(  
            "Lista cheia"  
        );  
    }  
  
    for (int i = tamanho; i > indice; i--) {  
        elementos[i] = elementos[i - 1];  
    }  
  
    elementos[indice] = valor;  
    tamanho++;  
}
```

COMO FAZEMOS O DESLOCAMENTO?

Valida se o índice não é maior que o tamanho

Verifica se o tamanho não é igual a capacidade

Reorganiza a lista



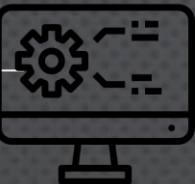
COMPLEXIDADE DA INSERÇÃO

Para deslocamento no início ou no meio

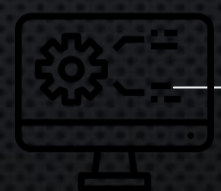
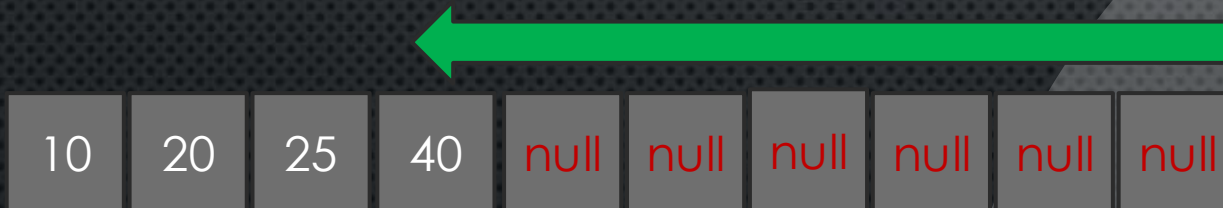
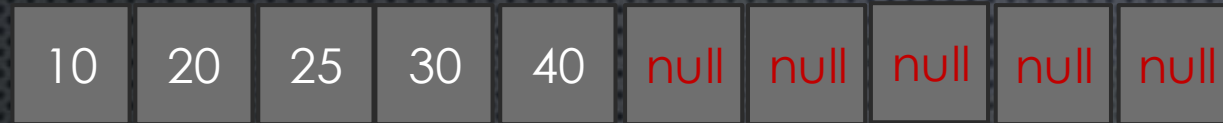
$$O(n)$$

Para deslocamento no final?

$$O(1)$$



REMOVENDO UM ELEMENTO



```
public int remover(int indice) {  
  
    if (indice < 0 || indice >= tamanho) {  
        throw new IndexOutOfBoundsException();  
    }  
  
    int removido = elementos[indice];  
  
    for (int i = indice; i < tamanho - 1; i++) {  
        elementos[i] = elementos[i + 1];  
    }  
  
    tamanho--;  
  
    return removido;  
}
```



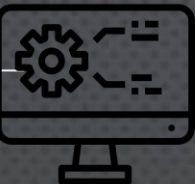
REMOVENDO
UM
ELEMENTO

LISTA CHEIA ???

- Isso significa que uma lista sequencial nunca pode crescer?
 - 2 soluções:
 - Implementar uma lista dinâmica
 - Aumentar a capacidade do array
-

EXERCÍCIO

- Faça o mesmo processo utilizando uma lista dinâmica



ATÉ A PRÓXIMA!

Continue estudando,
praticando e nunca pare
de evoluir!

NOSSA JORNADA
ESTÁ SÓ COMEÇANDO!

VALEU!

ATÉ
MAIS!

